

PROIECT DE LABORATOR: Color Quantization using K-Means

Disciplina: Procesarea Imaginilor
Universitatea Tehnică din Cluj-Napoca
An universitar: 2025 - 2026

1. Obiectivele proiectului

Conform enunțului,, proiectul are următoarele trei cerințe obligatorii:

1. Se vor studia metode de reducere a numărului de culori.
 2. Se va aplica algoritmul K-Means asupra pixelilor imaginii.
 3. Se va analiza efectul asupra reprezentării imaginii.
-

2. Studiul metodelor de reducere a numărului de culori

O imagine RGB pe 24 de biți poate conține până la 16.777.216 culori posibile ($256 \times 256 \times 256$). Reducerea numărului de culori (color quantization) este necesară pentru afișarea imaginilor pe dispozitive cu paletă limitată, compresie și segmentare.

2.1 Metoda Median Cut (Heckbert, 1982)

Cel mai popular algoritm de color quantization înainte de K-Means. Împarte spațiul culorilor în K regiuni cu număr egal de pixeli prin tăieri succesive la mediană.

Etapele detaliate:

- **Încadrarea:** Toți pixelii sunt plasați într-o "cutie" (bounding box) în spațiul RGB.
- **Selecția axei:** Se găsește axa cu cea mai mare lungime (R, G sau B).
- **Divizarea:** Pixelii sunt sortați după acea axă și se taie la jumătate, rezultând două cutii cu număr egal de pixeli.
- **Recursivitate:** Procesul se repetă pentru fiecare cutie până se obțin K cutii.
- **Reprezentarea:** Media fiecărei cutii devine culoarea reprezentativă.

2.2 Alte metode studiate

- **Population Algorithm:** Împarte spațiul RGB în celule egale și păstrează cele mai populate K celule (care cuprind cei mai multi pixeli).
- **Octree Quantization:** Reprezintă spațiul RGB ca un arbore cu 8 ramuri, fuzionând culorile similare când arborele depășește K frunze.
- **Reducerea pe histogramă (Lab 3):** Aplicată independent pe fiecare canal; poate genera combinații de culori care nu există în imaginea originală.

3. Implementare K-Means (Cod Sursă C++)

Algoritmul K-Means este implementat integral, fără a utiliza funcția predefinită `cv::kmeans()` din OpenCV.

3.1 Comparator pentru unicitatea culorilor

Pentru a garanta că centroizii inițiali sunt culori unice, definim un comparator pentru **std::set**.

```
struct ColorCompare {  
    bool operator()(const cv::Vec3b& a, const cv::Vec3b& b) const {  
        if (a[0] != b[0]) return a[0] < b[0];  
        if (a[1] != b[1]) return a[1] < b[1];  
        return a[2] < b[2];  
    }  
};
```

3.2 Inițializarea Centroizilor (Random Unic)

Această metodă asigură că cei K centroizi selectați aleatoriu sunt culori distincte extrase din imagine, evitând convergența către soluții slabe dacă centroizii inițiali ar fi prea apropiați.

```
vector<Vec3b> initializeazaCentroizi(Mat& img, int K) {  
    vector<Vec3b> centroizi;  
    std::set<Vec3b, ColorCompare> setUnique;  
    srand(42); // seed fix pentru reproductibilitate  
  
    while (centroizi.size() < K) {  
        int r = rand() % img.rows;  
        int c = rand() % img.cols;  
        Vec3b culoare = img.at<Vec3b>(r, c);  
  
        if (setUnique.find(culoare) == setUnique.end()) {  
            setUnique.insert(culoare);  
            centroizi.push_back(culoare);  
        }  
    }  
    return centroizi;  
}
```

3.3 Algoritmul K-Means Complet

```
Mat kMeansColorQuantization(Mat& img, int K, int maxIteratii = 10) {  
    vector<Vec3b> centroizi = initializeazaCentroizi(img, K);  
    Mat labels(img.rows, img.cols, CV_32SC1, Scalar(0));
```

```

for (int iter = 0; iter < maxIteratii; iter++) {
    // ASIGNARE
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            Vec3b pixel = img.at<Vec3b>(i, j);
            double distMin = DBL_MAX;
            int etichetaMinima = 0;
            for (int k = 0; k < K; k++) {
                double dist = distantaEuclidiană(pixel, centroizi[k]);
                if (dist < distMin) {
                    distMin = dist;
                    etichetaMinima = k;
                }
            }
            labels.at<int>(i, j) = etichetaMinima;
        }
    }

    // ACTUALIZARE CENTROIZI
    vector<Vec3d> sumaCanale(K, Vec3d(0, 0, 0));
    vector<int> numarPixeli(K, 0);
    for (int i = 0; i < img.rows; i++) {
        for (int j = 0; j < img.cols; j++) {
            int k = labels.at<int>(i, j);
            Vec3b p = img.at<Vec3b>(i, j);
            sumaCanale[k][0] += p[0];
            sumaCanale[k][1] += p[1];
            sumaCanale[k][2] += p[2];
            numarPixeli[k]++;
        }
    }

    vector<Vec3b> centroiziNoi(K);
    for (int k = 0; k < K; k++) {
        if (numarPixeli[k] > 0) {
            centroiziNoi[k][0] = (uchar)(sumaCanale[k][0] / numarPixeli[k]);
            centroiziNoi[k][1] = (uchar)(sumaCanale[k][1] / numarPixeli[k]);
            centroiziNoi[k][2] = (uchar)(sumaCanale[k][2] / numarPixeli[k]);
        } else {
            centroiziNoi[k] = centroizi[k];
        }
    }

    // CONVERGENȚĂ
    bool converged = true;
    for (int k = 0; k < K; k++) {
        if (distantaEuclidiană(centroizi[k], centroiziNoi[k]) > 1.0) {
            converged = false; break;
        }
    }
    centroizi = centroiziNoi;
}

```

```
        if (converged) break;    }

    Mat result(img.size(), img.type());
    for (int i = 0; i < img.rows; i++)
        for (int j = 0; j < img.cols; j++)
            result.at<Vec3b>(i, j) = centroizi[labels.at<int>(i, j)];
    return result;
}
```

4. Analiza rezultatelor

- **Calitate vizuală:** Cu $K = 16-32$ culori, calitatea vizuală este aproape aceeași cu aceea a originalului.
- **Eroare:** Pe măsură ce K crește, eroarea RMSE scade, dar timpul de procesare crește.
- **Concluzie:** K-Means produce rezultate superioare față de reducerea pe histogramă (Lab 3) deoarece lucrează în spațiul 3D.

5. Bibliografie

1. M. E. Celebi, "Improving the Performance of K-Means for Color Quantization", 2011 [cite: 13, 267, 333].
2. L. Brun, A. Tremeau, "On Color Image Quantization by the K-Means Algorithm", 2004 [cite: 20, 269, 334].
3. P. Heckbert, "Color Image Quantization for Frame Buffer Display", 1982 [cite: 25, 270, 335].
4. R. C. Gonzalez, R. E. Woods, "Digital Image Processing, 4th Edition", 2017 [cite: 29, 272, 336].